

Processus de compilation

1. Deux types de fichiers ?

En C++, on sépare ce qu'une chose est de ce qu'elle fait.

Le fichier d'en-tête (.h) annonce qu'une fonction existe, quels paramètres elle prend et ce qu'elle retourne. Nous n'avons aucune implémentation. C'est ce qu'on appelle une déclaration.

Le fichier source (.cpp) contient le corps des fonctions, entre accolades. C'est la définition.

Syntaxe	Nature
<code>extern int x;</code>	est une déclaration qui annonce qu'une variable x est définie ailleurs.
<code>int x;</code>	est à la fois une déclaration et une définition.
<code>void f();</code>	est une déclaration qui annonce que la fonction existe et décrit sa signature.
<code>void f() { }</code>	est une définition qui fournit le corps de la fonction.

`int x;` est à la fois une déclaration et une définition. La variable est définie, une adresse lui est assignée.

Cette équivalence entre « définir » et « réserver de la mémoire » ne vaut que pour les variables. Définir une fonction, c'est fournir son code et définir un struct, c'est décrire un type.

2. Les trois étapes de la construction

Le préprocesseur fait d'abord du ménage dans votre fichier. Quand il rencontre `#include "player.h"`, il copie-colle littéralement tout le contenu de `player.h` à cet endroit précis. Il traite aussi les `#define` et les conditions `#ifndef`. Le fichier « augmenté » qui en résulte sera ce que le compilateur va lire.

Le compilateur traduit ensuite cette unité de traduction et produit un fichier objet (.o ou .obj). Il compile chaque fichier .cpp séparément. Lorsqu'il compile `main.cpp`, il ne lit pas automatiquement les autres fichiers .cpp du projet. S'il rencontre un appel comme : `ajouterPoints(10);` et qu'il connaît la déclaration de `ajouterPoints`, il peut vérifier que l'appel est valide même s'il ne voit pas encore la définition de la fonction. Le fichier objet contient alors une référence vers `ajouterPoints` qui devra être résolue plus tard.

L'éditeur de liens (le *linker* combine les fichiers objets et les bibliothèques nécessaires. Il tente de relier chaque référence à sa définition réelle. Si une fonction ou une variable utilisée a été déclarée mais n'est définie nulle part, l'édition de liens échoue avec une erreur de symbole non résolu.

3. La règle de la définition unique

Pour les éléments ordinaires que nous utilisons dans ce cours, retenez surtout ceci : chaque fonction et chaque variable doit avoir exactement une définition.

C'est précisément pour ça qu'on ne met pas les définitions dans un .h. Souvenez-vous que `#include` fait un copier-coller : si trois fichiers .cpp incluent le même en-tête qui contient `void f() { ... }`, alors trois fichiers objets contiendront chacun leur propre copie complète de `f`. Le linker arrive, en trouve trois, et ne sait pas laquelle choisir. Il refuse de continuer.

Les templates qui sont des recettes échappent à cette règle. Leur définition doit donc rester dans l'en-tête. Les templates dépassent le cadre de ce cours.

4. Partager une variable entre plusieurs fichiers : `extern`

Une variable globale définie dans un .cpp est bel et bien exportée : le linker la voit et serait prêt à la donner à qui la demande. Le problème est ailleurs le compilateur des *autres* fichiers, lui, n'en a jamais entendu parler et refusera de compiler.

Il faut donc la lui annoncer, sans la recréer. C'est le rôle de `extern` :

```
// config.h
extern int score;    // un entier nommé score existe quelque part
```

```
// fichier.cpp
int score = 0;       // c'est ici qu'il est réellement créé
                    // sa définition doit être à portée globale
```

```
// autre.cpp
#include "config.h"

void test()
{
    score += 10;     // je peux utiliser score même si je ne l'ai pas défini ici
    (...)           // config.h m'a permis de voir sa déclaration
```

Tous les fichiers qui incluent `config.h` peuvent maintenant utiliser `score`, et le linker les reliera tous à la même variable, donc au même emplacement mémoire.

Pour les fonctions, `extern` est généralement inutile : les fonctions définies à portée globale ont déjà une liaison externe par défaut. Écrire : `void f();` ou `extern void f();` revient donc au même dans ce contexte. C'est pour cette raison qu'on voit rarement `extern` devant une déclaration de fonction.

Un piège à connaître. Pour une variable, dès qu'on ajoute une initialisation, la déclaration devient une définition : `extern int score = 0;` Cette ligne crée réellement la variable `score`. Ce n'est donc plus une simple déclaration `extern`.

5. Une variable locale qui survit : `static`

Placé devant une variable à l'intérieur d'une fonction, `static` ne change pas qui peut voir la variable, elle reste invisible en dehors de la fonction. Ce que `static` change, c'est sa durée de vie.

Normalement, une variable locale naît à l'appel de la fonction et meurt à la fin. Une variable `static` locale est initialisée une seule fois, au tout premier passage, et garde sa valeur d'un appel à l'autre.

```
void incrementer()
{
    static int compteur = 0;    // initialisée une seule fois
    ++compteur;
    std::cout << compteur << ' ';
}
// Trois appels affichent : 1 2 3
```

Elle n'est pas sur la pile, mais dans le segment de données du programme, comme une variable globale.

6. Décoder les messages d'erreur

Un message vous dit non seulement ce qui ne va pas, mais aussi à quelle étape ça a bloqué. C'est souvent l'information la plus utile.

Message	Étape	Cause habituelle
'x' was not declared in this scope	Compilation	#include oublié, faute de frappe
redefinition of 'x'	Compilation	Deux définitions dans le même fichier
undefined reference to 'f()' ou LNK2019	Édition de liens	Fonction annoncée et utilisée, mais dont la définition n'est pas trouvée
multiple definition of 'x' ou LNK2005	Édition de liens	Une définition placée dans un .h inclus par plusieurs fichiers

7. Les bonnes habitudes

Un seul critère permet de trancher pour à peu près tous les cas : est-ce que cette ligne crée quelque chose en mémoire ? Si oui, elle ne va qu'à un seul endroit, dans un .cpp. Si elle se contente de *décrire* sans rien créer, elle peut être recopiée dans autant de fichiers que nécessaire, donc elle va dans le .h.

Ce qui va dans le .h

Élément	Exemple	Pourquoi c'est sans danger
Prototypes de fonctions	<code>double norme(const Vecteur& v);</code>	Annonce seulement, aucun code produit
Définitions de struct et de class	<code>struct Vecteur { double x, y, z; };</code>	C'est un plan de construction, pas un objet
Alias de types (using)	<code>using Poids = double;</code>	Un simple nouveau nom pour un type existant
Énumérations	<code>enum class Couleur { Rouge, Vert };</code>	Même logique qu'un struct
Templates	<code>template <typename T> T max(T a, T b) { ... }</code>	Le compilateur doit voir la recette complète
Constantes globales	<code>const double PI = 3.14159;</code>	Voir la précision ci-dessous
Déclarations extern	<code>extern int score;</code>	Annonce une variable créée ailleurs

Les définitions de types ne violent pas la règle de la définition unique. Écrire `struct Vecteur { ... };` ne réserve aucun octet, mais explique au compilateur ce qu'est un Vecteur et combien de place il occupe. La règle autorise donc explicitement qu'un même type soit défini dans plusieurs fichiers, à condition que les définitions soient identiques. Noter que le copier-coller du `#include` par le préprocesseur le garantit automatiquement. Il en va de même pour un `using` ou un `enum`.

Attention toutefois : la garde d'en-tête reste obligatoire. Définir deux fois le même struct à l'intérieur d'un seul fichier demeure une erreur de compilation, et c'est précisément ce qui arrive quand deux en-têtes incluent tous les deux Vecteur.h.

Les constantes globales sont un cas particulier. En C++, une variable `const` déclarée hors de toute fonction est privée à son fichier par défaut — contrairement à une variable ordinaire. Écrire `const double PI = 3.14159;` dans un .h ne provoque donc aucune erreur multiple definition : chaque .cpp reçoit simplement sa propre copie. Pour des valeurs simples, le compilateur remplace même directement le nom par la valeur et n'utilise aucune mémoire. Il n'y a pas de piège en pratique, sauf si vous aviez besoin d'une adresse unique partagée — dans ce cas, revenez au couple habituel `extern const` dans le .h et définition dans un seul .cpp. C'est rare.

Notez la différence avec `int score;` sans `const` : celui-là est bel et bien partagé, donc dupliqué, donc interdit dans un en-tête.

Ce qui va dans le .cpp

- Le corps des fonctions
- Les variables globales, définies une seule fois dans tout le projet
- Tout ce qui est purement interne au fichier et que les autres n'ont pas à connaître

Deux réflexes à garder

Protégez systématiquement chaque en-tête, avec #pragma once ou la garde classique :

```
#ifndef VECTEUR_H
#define VECTEUR_H
// ...
#endif
```

Et ne mettez jamais `using namespace std;` dans un .h. C'est le seul using vraiment dangereux à cet endroit : chaque fichier qui inclut votre en-tête se retrouve avec tout l'espace de noms `std` déversé dans le sien, sans l'avoir demandé et sans moyen d'annuler. Les conflits de noms qui en découlent sont pénibles à diagnostiquer. Dans un .cpp, c'est acceptable ; dans un .h, écrivez `std::` au long.

Les variables globales en général

Utilisez-les avec prudence. Quand une valeur globale devient fausse, le coupable peut être n'importe quelle ligne de n'importe quel fichier du projet, et rien dans le code ne vous indique où chercher. Passer la valeur en paramètre demande deux lignes de plus à l'écriture, mais rend le chemin de la donnée visible et le débogage beaucoup plus court.

.

8. Le tout en trois fichiers

```
// player.h
#ifndef PLAYER_H
#define PLAYER_H

extern int score;           // déclaration
void ajouterPoints(int points); // prototype

#endif
```

```
// player.cpp
#include "player.h"
#include <iostream>

int score = 0;              // l'unique définition

void ajouterPoints(int points)
{
    score += points;
    std::cout << "Score actuel : " << score << '\n';
}
```

```
// main.cpp
#include "player.h"

int main() {
    ajouterPoints(10);
    ajouterPoints(50);
    return 0;
}
```

player.cpp donne player.o, qui contient le code de ajouterPoints et l'emplacement mémoire de score. main.cpp donne main.o, qui contient un trou à remplir pour ajouterPoints. Le linker fait le pont entre les deux.

Le test : si vous effacez la ligne `int score = 0;` de player.cpp, à quel moment ça casse ? Les deux fichiers compilent sans broncher — le `extern` du .h suffit à satisfaire le compilateur. L'échec arrive à l'édition de liens : ajouterPoints réclame un score qu'aucun fichier objet ne fournit.